Moca template lifecycle automation

Self-serve sandbox templates, gated by policy, with provenance

sbx-template · 2026-11-09

Outline

6 sections; the numbers are slide numbers

1 Why and what

Moca's motif, the problem, the design in four ideas

3 The brain thinks, the hands act; a template shapes the hands
4 Benchmarks need many templates; adding one was manual
5 Self-serve: any enrolled client builds; policy, not a person, decides

2 Architecture

where each part runs, and a build's states

6 One control plane decides; hosts build and run; clients only ask
7 A build has two states: how far from usable, and whether it runs now

3 Onboarding

create to registered, and the refusals

8 Create to registered: no approver, every decision recorded
9 Refusals are answers with a rule, not silence

4 Using a template

activation, first use, sessions, one plane, every route

10 Activation is a lease; the worker starts in seconds
11 First use: name a template and go
12 A Moca Pi session runs only on its build
13 One control plane serves both kinds of brain
14 The API covers the whole lifecycle, create to retire

5 Provenance and validation

records chained by digest; windows 1-4; create to retire

15 Every step is a chained record, checkable offline
16 Four windows validated the lifecycle end to end
17 Create to retire, through the API alone, on the cluster
18 Activation fell from ~100 s to ~11-17 s

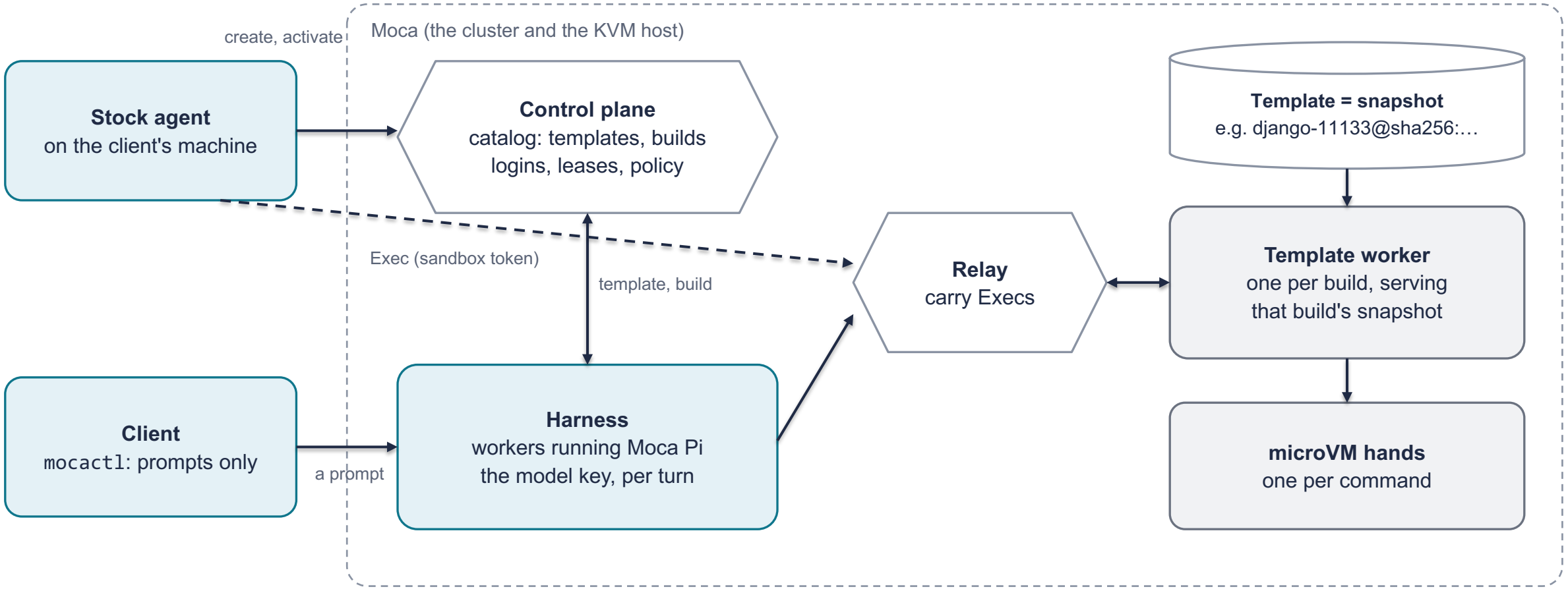
6 At scale, and next

Lite-28; two brains on one build; findings; the open gaps

19 SWE-bench Lite on templates, in groups, retire between
20 Two brains on one build, every link exact
21 What the windows found, and the fixes
22 Next: egress records, then capacity, then upstream

The brain thinks, the hands act; a template shapes the hands

Moca Pi brain, or a stock agent, with Moca microVM sandbox hands

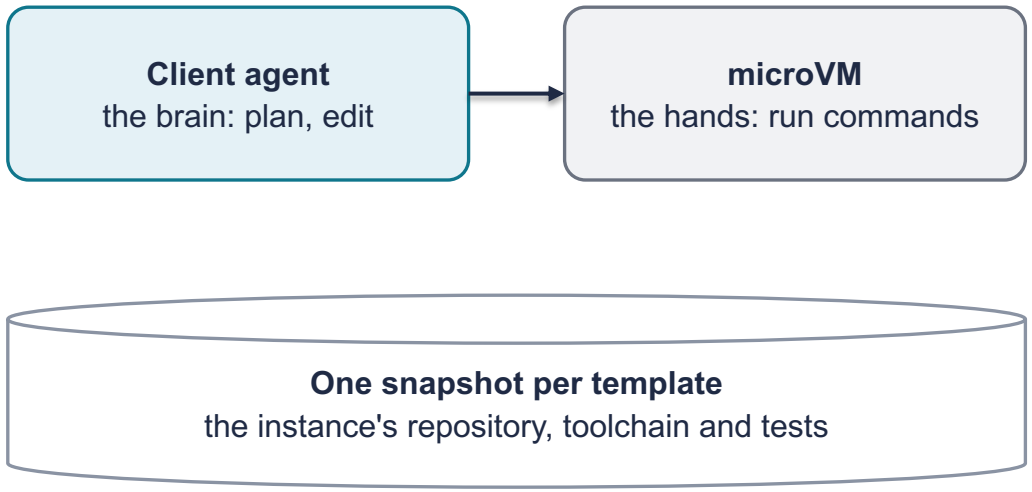


The hands never hold the model key or the agent
they get commands and return output; a template decides what the hands contain

Benchmarks need many templates; adding one was manual

Where the work started (spec §1-§2)

- **One template = one SWE-bench instance's environment**, as a microVM snapshot that every command of that instance runs in.
- **Before:** each template was a hand-run build, a hand-picked worker id, a settings edit and a redeploy of the control plane.
- **At scale:** 30+ per-instance templates, built and retired as experiments run, by clients that are not operators.
- **And proof:** which image, which checks, which snapshot served a given run.



The goal (the user, 2026-11-02)
SWE-bench at scale on templates, self-serve and gated, validated end to end before any upstream PR

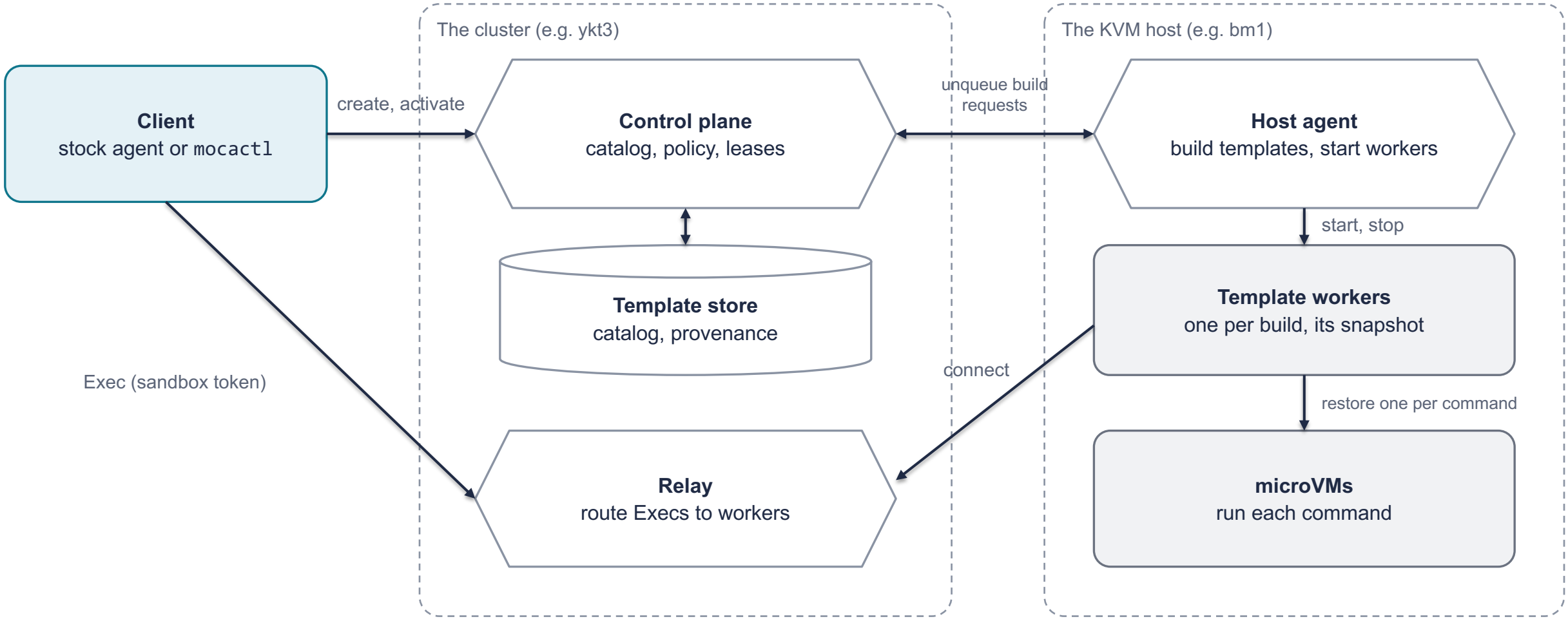
Self-serve: any enrolled client builds; policy, not a person, decides

The design in four ideas (spec §3, §6-§8; D12-D13)

Templates are declared in a store	a template is a name over immutable builds; a build's id is its snapshot's manifest hash
Policy gates every image	machine-checked rules at create and at registration; a `benchmark` profile needs a trusted gate record
Hosts build and run; the control plane decides	a host agent claims builds and starts workers; clients activate by lease, within quotas
Every step is a chained record	append-only provenance named by digest, verifiable offline with no access to the cluster

One control plane decides; hosts build and run; clients only ask

Who runs where: the cluster (OpenShift; e.g. ykt3) and the KVM host (bare metal; e.g. bm1)



A worker **connects** to the relay and holds one two-way gRPC stream (Attach): commands come down it, results go back up. Readiness is a worker connected to the relay; the cluster holds no VM, the KVM host no policy.

A build has two states: how far from usable, and whether it runs now

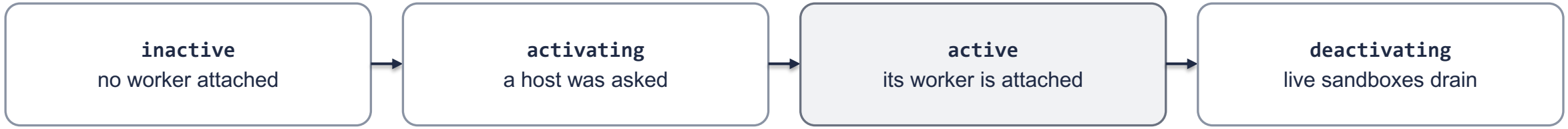
Onboarding is set by the host's reports and policy; readiness by what is attached

Onboarding (spec §3.3)



Off the path: `failed` (a build or probe failure, with its step; or the owner's cancel), `rejected` (policy at registration, with its rule), `retired`.

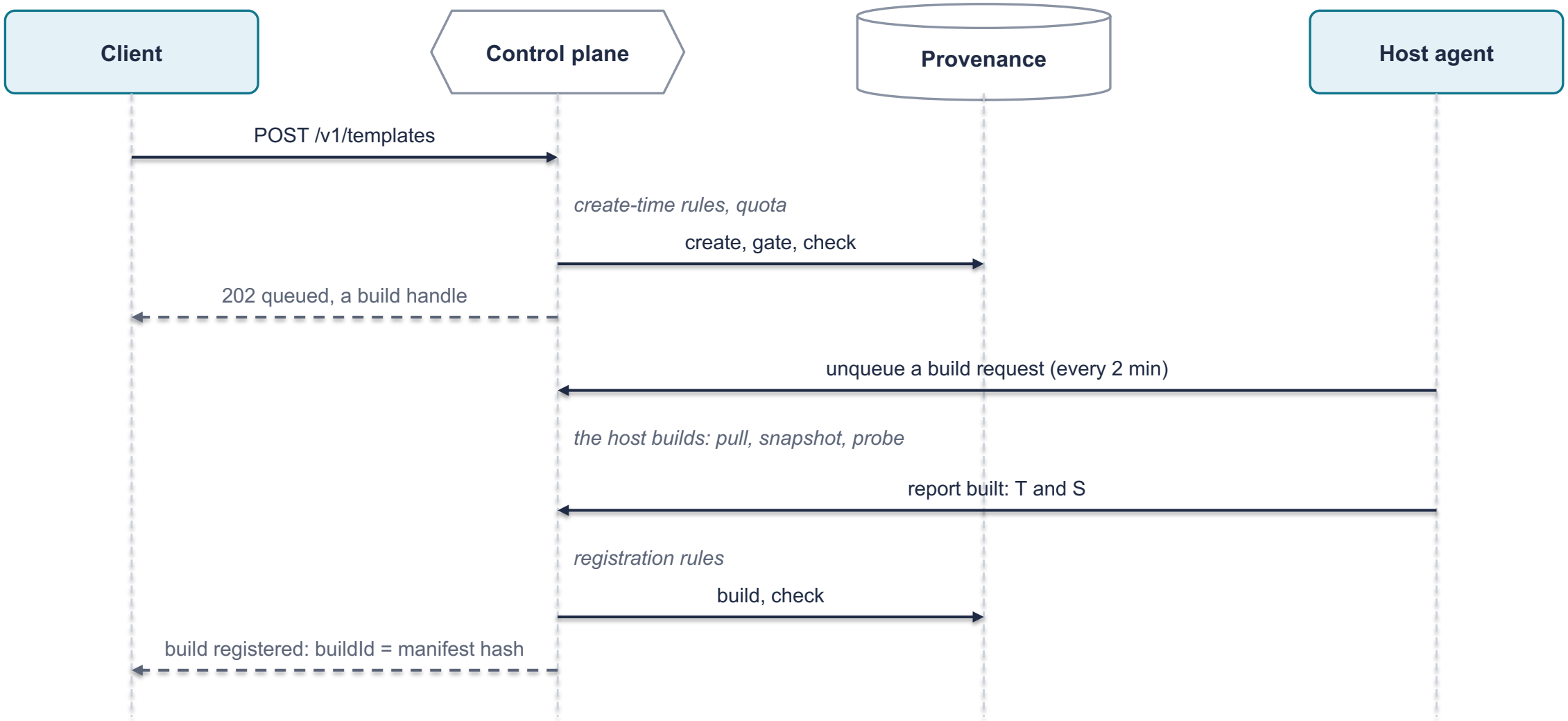
Readiness of a registered build



`unavailable`: no host holds its snapshot, or every holder is full. `active` is derived from the relay's attached record, so the state cannot claim a worker that is not there.

Create to registered: no approver, every decision recorded

Sequence 1: onboarding (spec §4, §6.1, §8)



Refusals are answers with a rule, not silence

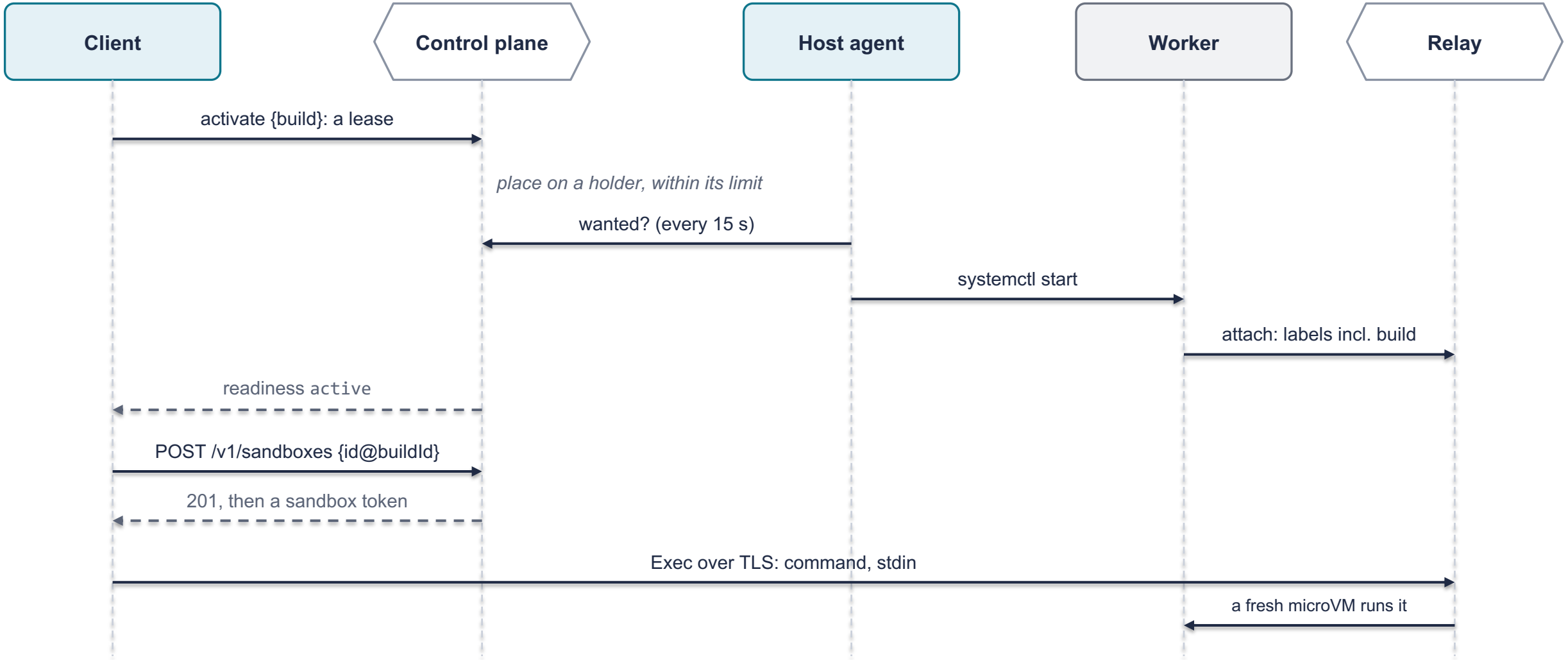
What a client sees when a create cannot proceed (spec §4, §8; G21, G22, G29)

Case	Answer	Recorded
Policy refuses the create	422 `policy_refused` with the rule id	a `check` record
Another subject owns the id	409 `template_exists`	nothing queued
Same inputs as a build a host still holds	200 or 202 with `reused: true`	no second build
Over a quota	429 `quota_exceeded`, naming which	nothing queued
The owner withdraws a build	cancel: `failed` at step `cancelled`	a `build` record
The build or probe fails on the host	`failed`, with step and code (`probe_failed`)	a `build` record

Nothing that names where an image lives leaves the host
failure text is redacted; a record naming the registry is not sent at all (G25, G28)

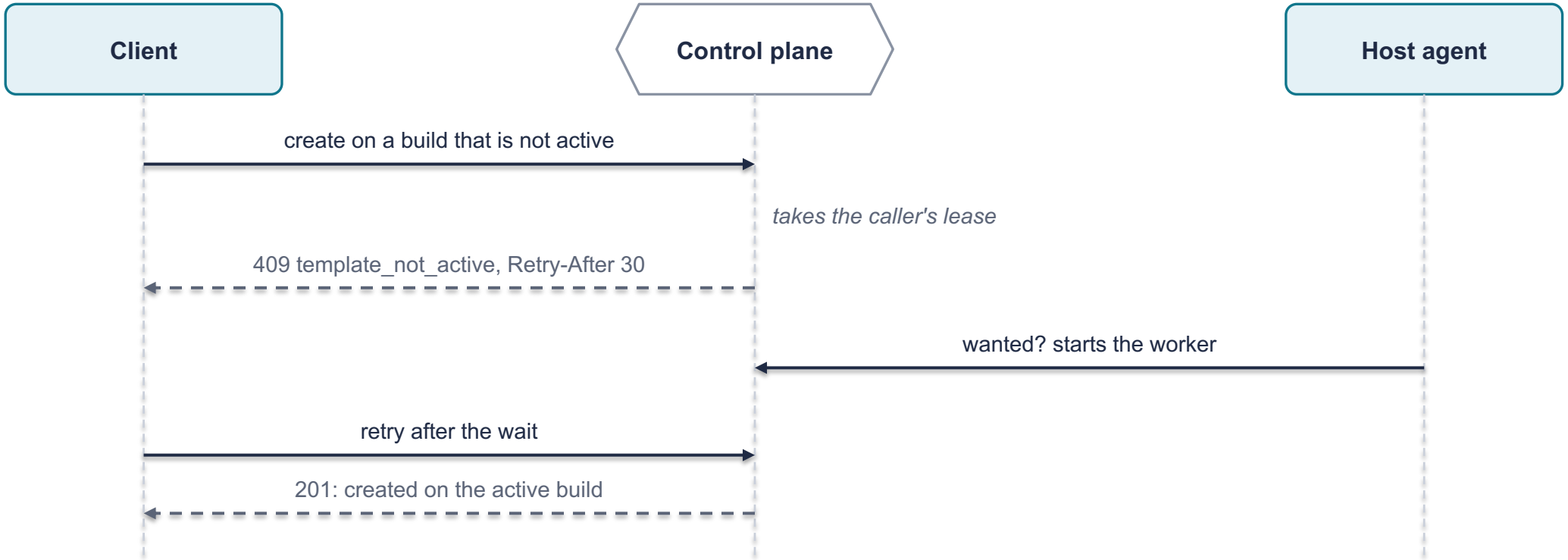
Activation is a lease; the worker starts in seconds

Sequence 2: activate, then a sandbox on the pinned build (spec §6.3; G31)



First use: name a template and go

Activation on first use (spec §6.4), for sandboxes and sessions alike

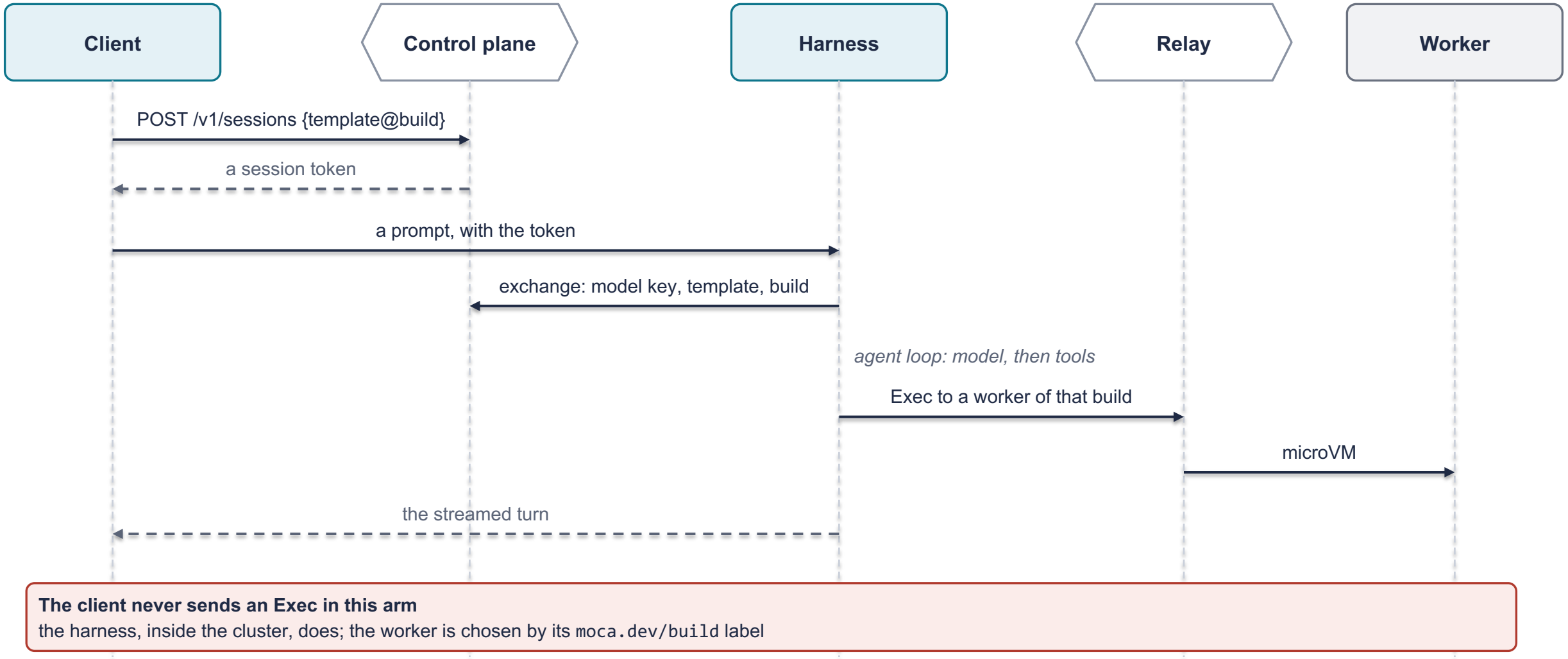


`activate: false` starts nothing. No sandbox is kept while waiting, so a wait never uses the caller's sandbox cap. `mocactl` retries for up to 10 minutes, saying why.

Without host agents nothing changes: the create succeeds as before

A Moca Pi session runs only on its build

Sequence 3: a session pinned to a build (combined control plane §3.4; G19)



One control plane serves both kinds of brain

The combined route profile (combined control plane design, C1-C4)



Decision	Chosen
C1 How	a third profile, `combined`; `harness` and `sandbox` unchanged
C2 Login	open as on `harness`; enrollment checked on sandbox routes and template writes
C3 Where	a new namespace, so the validated SBX0 stays as it is
C4 Turns by build	in: a worker label and a filter (G19)

The API covers the whole lifecycle, create to retire

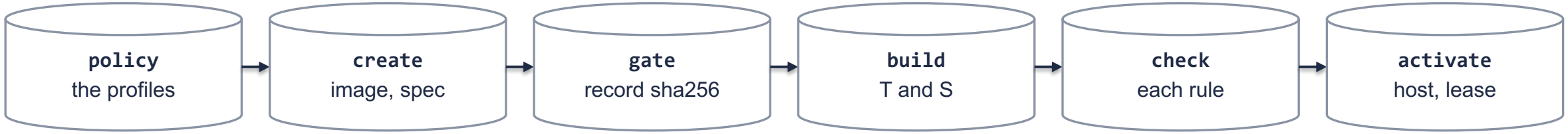
Each activity a route and a mocactl subcommand; each writes a record naming who

Activity	Who	Record it writes
create a template, or a new build	any enrolled client	create, gate, check
build, probe, register by policy	the host agent, then policy	build, check
activate, deactivate (a lease)	the owner; anyone, if shared	activate, deactivate
move a tag onto a new build	the owner	tag (new and old build)
edit description, visibility	the owner	visibility (new and old)
retire a template or a build	the owner	retire (and the host's removal)
reject a build after the fact	an admin, with a reason	reject
transfer ownership	an admin	owner
set quotas, default or per user	an admin	policy

Deployed and shown end to end
a retire never cuts off running work: refused while a sandbox or session uses the build

Every step is a chained record, checkable offline

Provenance (spec §7): digests, not signatures



- **One file per record**, named by the sha256 of its canonical JSON; each names its predecessor in `prev`, with a `seq`. Any edit, drop or reorder breaks the chain.
- **Stored in the control plane** on a volume, not Redis: records, a head per stream, and the cited gate records byte for byte.
- **Verified offline** by a stdlib Python script or the same rules in TypeScript, from an export, on a machine with no access to the cluster.

An export proves itself consistent, not complete
compare each stream's head with a copy kept elsewhere to catch a dropped tail

Four windows validated the lifecycle end to end

Spec §10 steps, as run 2026-11-04 to 2026-11-05 (no model)

Step	Window 1	Window 2	Windows 3-4
Create, build, register	all four	all four, fresh	all four, each
Refused create (X10)	yes	yes	yes
Broken probe	not reached	`probe_failed`	step `probe` (w4)
First use	not built	409, then active	as w2
Served build = pinned	every sandbox	every sandbox	every sandbox
Patches match recordings	not compared	every patch	every patch
Offline verification	83 records	140 records, all chains verified	not repeated

Step 3 (a second client on the same instance) passed separately with a second enrolled subject: its create and its new build `409`, its cancel `403`.

Create to retire, through the API alone, on the cluster

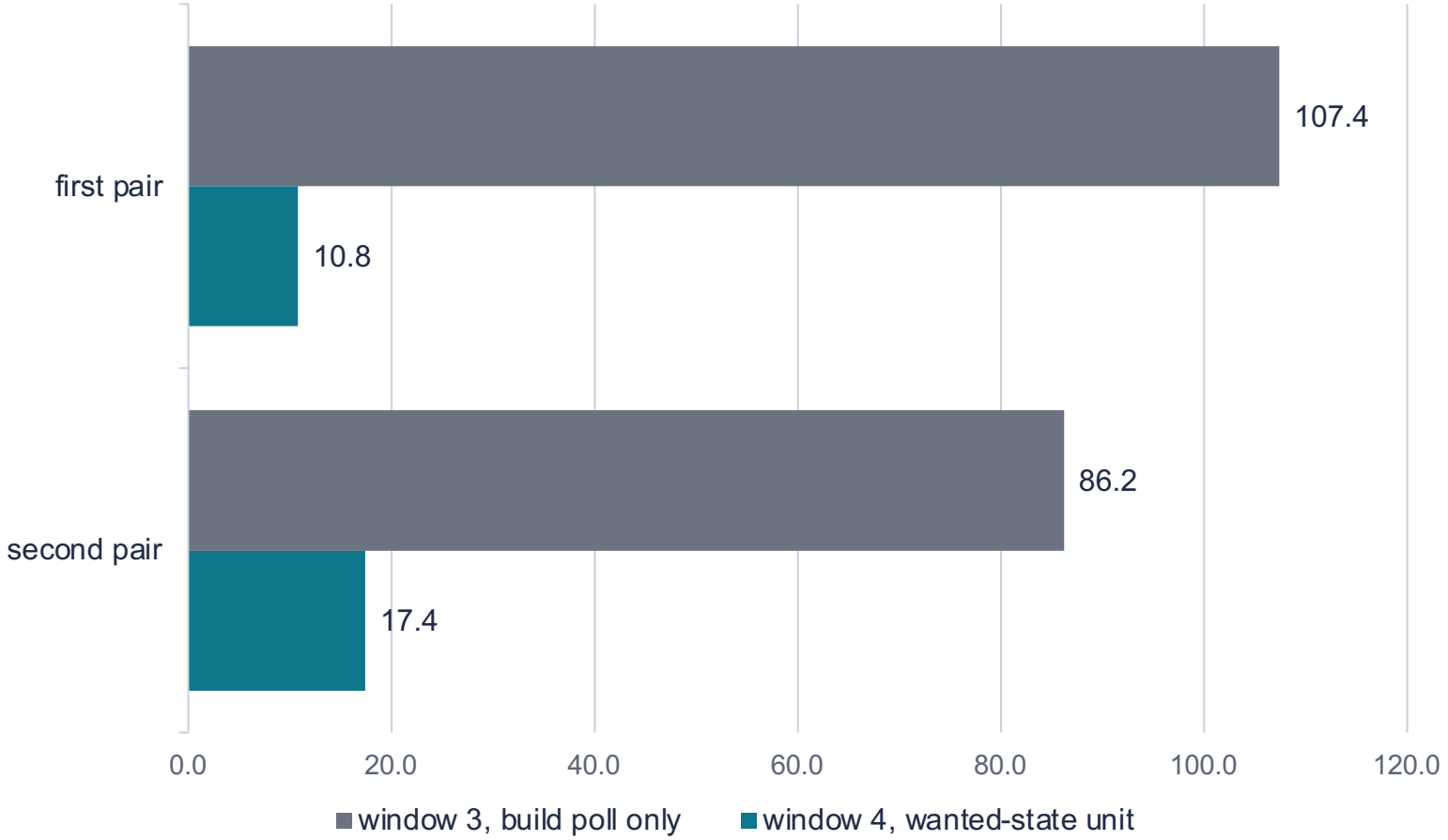
The lifecycle window, 2026-11-06: a user with no role; no operator step (no model)

Stage	What happened
create, build, register	all four by policy; the bad gate refused (X10); the bad probe failed
activate, first use, use	a lease in 23-26 s; every sandbox on its pinned build
retire	every build; a later create refused `410`; readiness `retired` in ~1 s
the host removes it	the KVM host's agent, by itself; `heldBy` emptied in 17-32 s
provenance	every chain before is an unchanged prefix after; 84 records appended
composition	an independent composer: every link of each rep unique

A whole-template retire is every build
earlier windows' builds of those templates were retired too: a new build is the way back

Activation fell from ~100 s to ~11-17 s

Seconds from each activation's request to `active`, both builds of a pair together (G31)



Before: an activation waited for the build poll, which starts 2 minutes after the previous one ends, and behind any build.

After: a separate unit applies the wanted state every 15 s; a lock keeps it and the build poll from acting at once.

One pair per window
a measurement, not a distribution

SWE-bench Lite on templates, in groups, retire between

Lite-28 on templates, 2026-11-06/07: stock brains, the sandbox profile, a real model

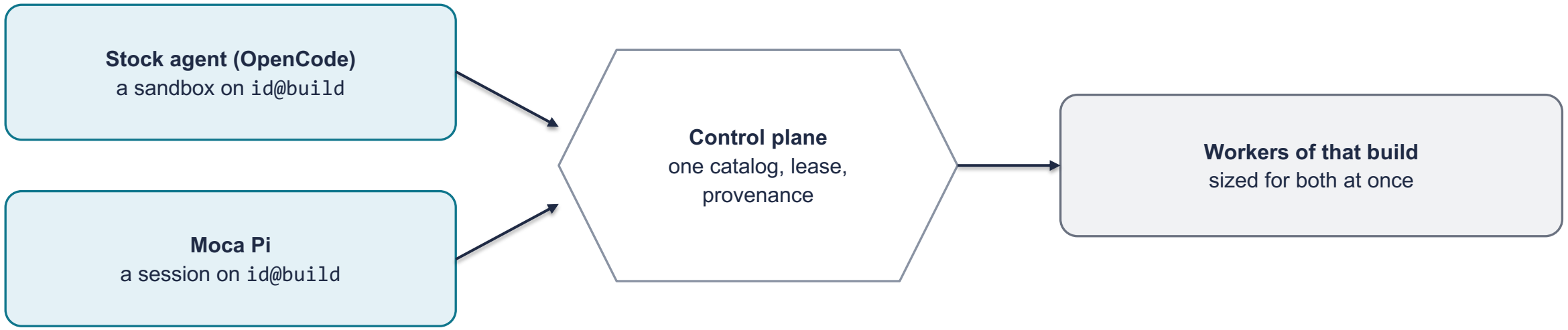
- **Self-serve, in three groups:** each group created, built and registered by policy, used on pinned builds, then retired; the host agent's removal checked before the next group.
- **Provenance:** every chain before an unchanged prefix after; an independent composer joined every link of every rep uniquely.
- **One instance left out:** its id is a template the deployment declares, which blocks a self-serve create under that id (G34).

Two agents on one template at once
the one-session minimum budget refused some commands at admission (G36): clients now retry and exit 75; budgets sized for the work run at once

Removal keeps workspaces
for the janitor; without it they stay on the host's disk

Two brains on one build, every link exact

The Moca Pi window, 2026-11-07: a stock agent and Moca Pi, a real model, the combined control plane



What the session now does	Seen
seeded from its build's own spec	`sandbox.seed.source: build`, the build = the pin
returns the workspace's patch with the turn	whole, its sha256 and size matching
leaves a `session` record at create and delete	its lease and build, on the template's chain
composes like a sandbox	every link of every rep unique, for both brains

What the windows found, and the fixes

Every finding has a register entry; Moca's are fixed and validated

Id	Found	Fixed in	Validated
G26	commands over 32 KiB hung the worker	the worker: refused at once	w2-w4
G25, G28	a registry host left the host in a reason	host agent redaction	w2-w4
G21, G29	duplicate builds; reuse of a removed snapshot	control plane reuse rules	w2-w4
G31	activation waited for the build poll	the wanted-state unit	w4
G19	a session could run on another build	the build label and filter	combined
G33	a rep's lease found only by time	sandbox records by digest	lifecycle
G35	a session got no seed, no patch, no record	seed from the build; patch; `session`	Moca Pi
G36	commands refused at a one-session budget	client retry; budget sizing	Moca Pi
G30	~20 s client stalls	the client's DNS (not Moca)	w4: 0 stalls

Since fixed
G34: an admin hands a declared template to a subject; G37: mocactl's template option takes `id@build`

Next: egress records, then capacity, then upstream

In order, each with the user's decision first

1. **Egress records through the API (G42)** — a sandbox's owner reads which hosts it reached; its end record names the log's digest
2. **Capacity by host resources (G39, G43)** — placement weighs memory and paces cold starts, from the batch window's measurements
3. **Propose upstream** — as reviewable PRs, from what the windows proved; upstream has no session templates today

Done since
users and roles through the API (G38); general-user self-serve with its own gate records (G41); a Docker Hub repository allowlist